



I need to improve the runtime of the attached MIP model without altering any of its functionality or features. What techniques can I use to achieve this, such as reformulating the constraints? What kind of problem class is the current model?

This model is a large-scale Gurobi MILP for daily (and multi-day "design") electric heavy-duty fleet disposition. It combines a set-partitioning/assignment core ($z[m, f, s]$ binaries covering every trip exactly once) with vehicle-routing flow-balance (route start/route end/chain), an intraday battery/SoC and V2G energy-scheduling layer, crew/driving-time tracking via cumulative big-M counters, and (in the design variant) a fleet-sizing binary layer. Below is the problem classification and concrete, functionality-preserving speedups, ordered roughly by expected impact.^[1]

What class of problem this is

Structurally this is a combined Electric Vehicle Scheduling Problem (E-VSP) with time windows and vehicle chaining, fused with an intraday energy/unit-commitment-style sub-problem (SoC, charging/V2G, PV self-consumption) and, in the design run, a fleet-sizing/facility-sizing layer on top. It is a mixed-integer *linear* program (all costs are linear, SoC-weight tangents are linearized via cutting planes), but it is dense in binaries: one $z[m, f, s]$ per vehicle $\times$ trip/event $\times$ feasible start time, plus per-timestep binaries for routing, breaks, and charging gating. This combination (assignment + time-indexed flow + big-M counters) is exactly the family known to blow up branch-and-bound trees, so it's NP-hard in practice and highly sensitive to formulation tightness, not just solver settings.^[1]

Biggest quick win: an unconditional `model.tune()` after every solve

In `solvemodel`, every single `model.optimize()` call is immediately followed by `model.write(...tuningmodel.lp)`, `model.tune()`, `model.getTuneResult(0)`, and another `model.write(...)`. Gurobi's auto-tuner by default re-solves the model several times (multiple parameter trials) to search for a better parameter set — this can multiply total runtime by several times on every single scenario/day/iteration, including inside multiprocessing sweeps. Since this doesn't change the model's constraints or objective, gating it behind an explicit flag (run once offline to harvest a `.prm` file, then just load that file with `model.read()` at the top of `solvemodel` for production runs) removes pure overhead without touching functionality at all. This is very likely your single largest lever.^[1]

Tighten the big-M formulations

Several constraints use one **global, horizon-wide big-M** where a tighter, index-specific bound would shrink the LP relaxation and cut B&B nodes dramatically, with zero change to the feasible set of integer solutions:

- The chaining timing constraint uses `bigmtime = laststep + max(tripsdurationsteps) * 2` uniformly for every (m, f, g) pair, even though the true worst-case gap for each specific pair (f, g) is much smaller (bounded by the horizon minus that trip's own duration/window). Compute a per-pair `bigM[f, g]` instead of one constant. ^[1]
- The driving-time/crew counters (`counter <= previous+drivennow`, reset via `-bigmsteps*atdepot`) reuse `bigmsteps = horizon*2` everywhere. Since `counter` is already bounded by `maxdrivesteps/horizon`, replacing these hand-rolled big-M pairs with **Gurobi indicator constraints** (`model.addGenConstrIndicator`) on the `atdepot` binary lets Gurobi's presolve derive the tightest possible bound automatically and is usually both faster and numerically more stable than a manual big-M. ^[1]
- `breakactive[m, t]` and `penalizedexternal[m, t]` are fully determined (via equalities) by sums of mutually-exclusive binaries. These can often be **substituted as linear expressions directly in the objective/constraints that use them** rather than kept as separate variables tied down by extra constraints — same feasible region, fewer variables and rows. ^[1]
- Repeated `model.getVarByName(...)` calls inside loops when linking the design model to per-day submodels force Gurobi to resolve names through its internal index each call; caching the Var objects in a Python dict while building avoids this build-time cost (this affects model-construction time, not solve time, but on large multi-day design runs it adds up). ^[1]

Solver parameters and warm starts (no formulation change)

- `MIPFocus` is currently 0 (balanced) — since your objective/bound behavior is unclear, try `MIPFocus=1` if the solver struggles to find any good feasible incumbent, or `MIPFocus=3` if it's stuck proving the gap on a good incumbent it already has.
- `Cuts`, `Presolve`, and `Aggregate` are all left at auto (-1/1); for big-M-heavy MILPs, explicitly trying `Presolve=2` (aggressive) and `Cuts=2` or `3` often helps because presolve can exploit the big-M structure better than "auto" guesses. ^[1]
- You already have a partial warm-start mechanism (solve a crew-relaxed model first, then feed its solution as `.Start` values into the crew-constrained model). The same idea can be extended across your scenario/year/V2G sweep iterations — since consecutive scenarios often differ by only one parameter, seeding each new MILP with the previous iteration's solution as a `MIPStart` should cut re-solve time substantially without altering results. ^[1]
- The `Symmetry` parameter is not set; given that identical vehicle "copies" exist in the fleet pool and the design run already breaks symmetry for `owned[m]` via `owned[earlier] >= owned[later]`, setting `Symmetry=2` (aggressive) globally, and extending an analogous lexicographic-ordering constraint to `z[m, f, s]` among interchangeable same-type vehicles in the *daily* model (not just the design model), removes a major source of redundant, symmetric branch-and-bound exploration. ^[1]

Structural speedups for the biggest instances

For the multi-day **design** run, all days are folded into one giant combined MILP sharing only the `owned[m]` binaries and the ownership objective term. This is mathematically equivalent to, but typically far slower than, a **Benders (or Lagrangian) decomposition**: an outer master problem picks `owned[m]`, and each day's scheduling subproblem is solved independently (in parallel, which your codebase already supports via `multiprocessing`) given a fixed fleet, feeding optimality/no-good cuts back to the master. This preserves identical functionality and the identical optimal answer while letting you exploit the day-by-day parallelism you already built for scenario sweeps, and avoids solving one enormous combined MILP with a huge shared search tree. ^[1]



1. `hdv_disposition_optimization.py`